Information Extraction

Oana Balalau

Inria, Ecole Polytechnique
oana.balalau@inria.fr

27/03/2026

Motivation: Machine Knowledge

Much of the knowledge we have as humans is stored as **text**.
Many of your favorites books are just collections of sentences!
Can we transform these sentences in edges in a graph? Yes!
(almost)

Motivation: Machine Knowledge

Data models such as RDFs are easy to use by algorithms, especially compared to text. Constructing very large RDFs is time consuming.

Information extraction methods enable extracting structured content from digital text documents.

Structured content can be used to automatically populate knowledge bases / knowledge graphs (KB, stored as RDF graphs): machine-readable facts about the real world.
Wikidata is a knowledge base / knowledge graph.

Information extraction

Information Extraction: extract structured data from unstructured (typically text) or semi structured machine-readable information (e.g. tables, json etc).

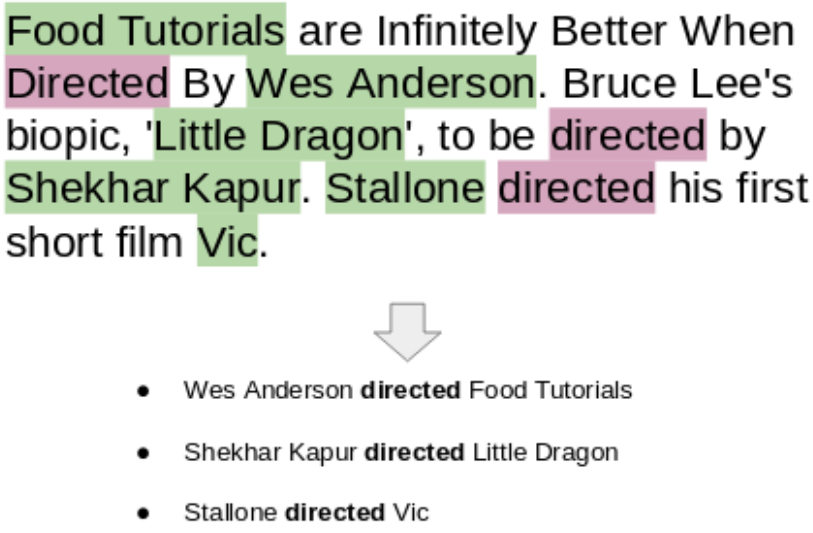


Figure: Extracting triples of the form (subject, predicate, object)

Information extraction tasks

Auxiliary tasks:

- sentence segmentation
- tokenization
- lemmatization
- part-of-speech tagging
- dependency parsing

Tasks useful for creating a KB:

- named entity recognition
- coreference resolution
- entity canonization (named entity linking)
- relation extraction / fact extraction
- relation canonization

Sentence Segmentation & Tokenization

Dr. Marie Curie discovered radium. She was married to Pierre Curie, and the couple resided in Paris.

- **Sentence segmentation:**
  1. Dr. Marie Curie discovered radium.
  2. She was married to Pierre Curie, and the couple resided in Paris.
- **Tokens (Sent 1):** ["Dr.", "Marie", "Curie", "discovered", "radium", "."]
- **Segmentation Method:** Split by punctuation (".?!") combined with a predefined list of abbreviations (like "Dr.", "Ms.") to avoid false splits.
- **Tokenization Method:** Split the text by spaces and punctuation marks.

Lemmatization

She was married to Pierre Curie..."

Lemmas:
"was" $\rightarrow$ *be*;
"married" $\rightarrow$ *marry*;
"resided" $\rightarrow$ *reside*

The standard form of a word is the **lemma** and the method is lemmatization. One simple approach is to search for the lemma of a word in a dictionary, such as WordNed.

Part-of-speech (POS) tagging

Figure: We label text with POS tags, using `https://parts-of-speech.info/`

Part-of-speech tagging

She will **race** you for it.
When will the **race** start?

On average a word can have 2 part of speech tags.
90% accuracy by choosing the most frequent tag for a word.

For even better performance we can use the context:

1. **rule-based**: create a large database of hand-written disambiguation rules, e.g. specify that a word is a noun rather than a verb if it follows a determiner (e.g. *the race*).

2. **probabilistic**: resolve tagging ambiguities by using a training corpus to compute the probability of a given word having a given tag in a given context.

Dependency parsing

Dependency parsing is the process of determining the syntactical structure of a sentence. Syntax refers to the arrangement of words and phrases in a sentence, finding multiple words that act as a single unit.

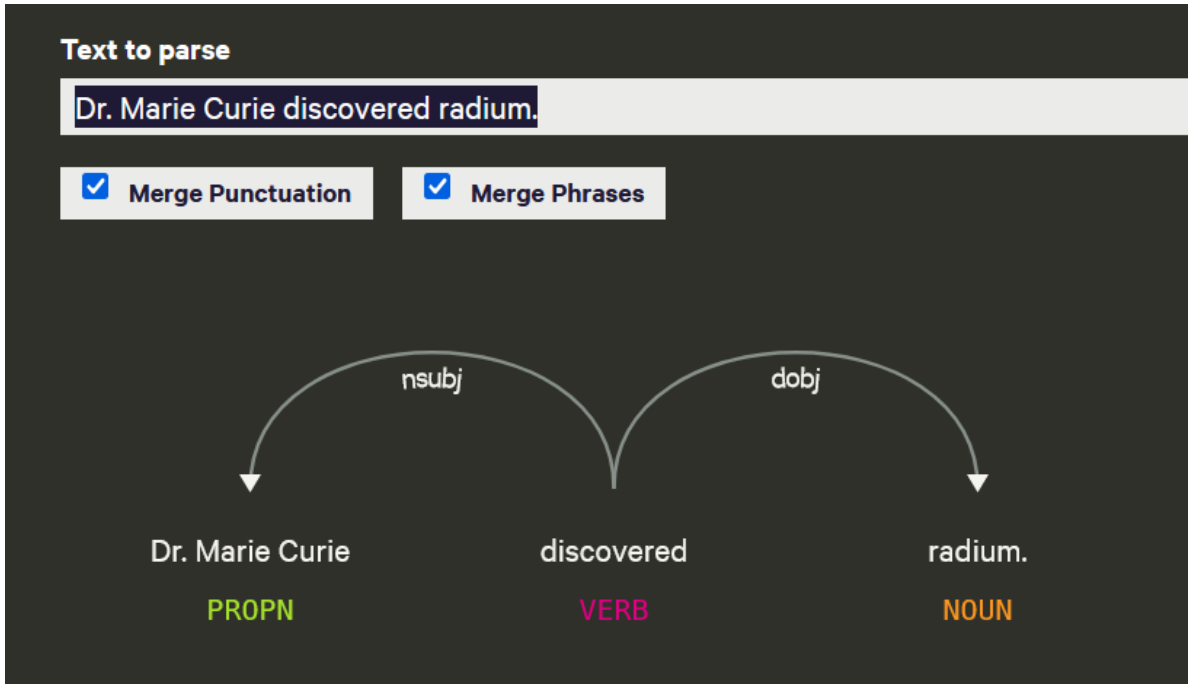


Figure: We label text with dependency tags tags, using `https://demos.explosion.ai/displacy`

Information extraction tasks

Auxiliary tasks:

- sentence segmentation
- tokenization
- lemmatization
- part-of-speech tagging
- dependency parsing

Tasks useful for creating a KB:

- named entity recognition
- coreference resolution
- entity canonization (named entity linking)
- relation extraction / fact extraction
- relation canonization

Named entity recognition (NER)

Dr. Marie Curie discovered radium. She was married to **Pierre Curie**, and the couple resided in **Paris**.

A named entity refers to a **person, location, organization, artifact** with a name.

NER Method: Assume all words starting with capital letters are entities, or check words against a trusted dictionary.

The first word in a sentence is capitalized.
Capital Letters can be used to mark Importance.
in informal text, such as tweets, there is no rule

Named entity recognition (NER)

First solution: all the words starting with capital letters are entities. Problems: beginning of sentences, prepositions in names (The Hunchback of Notre Dame), informal text with no/poor capitalization.

Second solution: have a dictionary of named entities. Problem: will not find entities that do not belong to the dictionary.

Third solution: Annotate text with the entities it contains and train a neural network.

A bit on regular expressions

What if we have identified words following a pattern and we would like to automatically extract them?
E.g. names, dates (DD-MM-YYYY), email addresses, or high level annotations such as POS tags.

A **regular expression** finds words over an alphabet (the alphabet can be the set of unicode characters, or a set of labels).
A language is a set of words: for example a, aa, aaa, aaaa, etc.
A **regular expression** describes a language.

A bit on regular expressions

$L1 = \{, a, aa, aaa, ...\} \rightarrow a*$
'*' (asterisk) matches zero or more of the preceding character.

$L2 = \{ab, abc, abcc, abccc, ...\} \rightarrow abc*$

$L3 = \{, ab, abab, ababab, ...\} \rightarrow (ab)*$
parentheses are used to group regex letters together, () is a group

$L4 = \{0, 1, 2, 3, 1, 5, 6, 7, 8, 9\} \rightarrow (0|1|2...|9) = [0-9]$

$L5 = \{Albert\ Einstein, Harry\ Potter, ...\} \rightarrow [A-Z][a-z]+\ [A-Z][a-z]+$
'+' (plus) matches one or more of the preceding character

$L6 = \{a, ab, abb, bbba, aaabbab, ababa, ...\} \rightarrow (a|b)+$

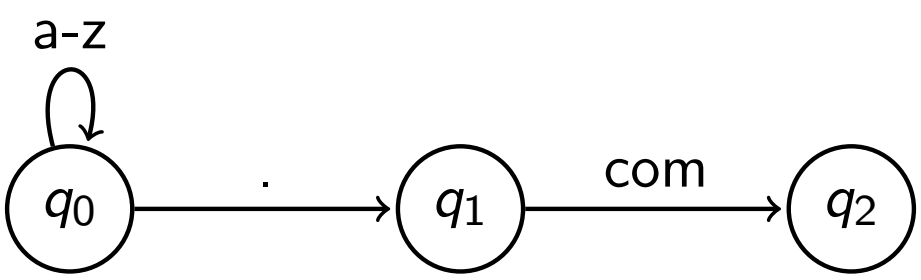
$L7 = \{ac, b, aa, bbc, aaa, bbb, ...\} \rightarrow (a+|b+)c?$
'?' (question mark) means match zero or one instance of this character

Advanced NER: Regex Under the Hood

We know Regular Expressions are used to extract patterned text. But how does the computer actually execute them?

They are compiled into **Finite State Machines (FSMs)**.

- An FSM is a directed graph where nodes are "states" and edges are character transitions.
- A string is accepted if a path from the "Start" state leads to a "Final" state.



Example FSM accepting strings ending in ".com"

NER with Dictionaries: The Complexity Problem

We established that using a dictionary of known entities is a good baseline for NER. But how do we actually execute the search?

The Naïve Approach

Imagine a Knowledge Base with $M = 10,000,000$ entities (like Wikidata) and a text document of length $N = 1,000$ words.

- If we scan the text for *every* entity in the dictionary, the time complexity is $O(N \times M)$.
- This requires **10 billion string comparisons** for a single short text!

The Bottleneck: Standard data structures like flat Lists or Arrays force us to do linear searches. Even Hash Sets struggle here because we have multi-word entities (like "New York University") and we don't know the exact entity boundaries.

Efficiently finding words in a data structure: Trie

A trie (pronounced as tree), also called a digital tree or a prefix tree, is a tree where nodes are boolean and edges are labeled with letters.

A string **S** is contained in the trie if S denotes a path from the root to a node labelled true.

Example: Ada, Adams, Adora.

To add a node that is a prefix of an existing string, switch the node to true. To add a string with no common prefix, create a new branch.

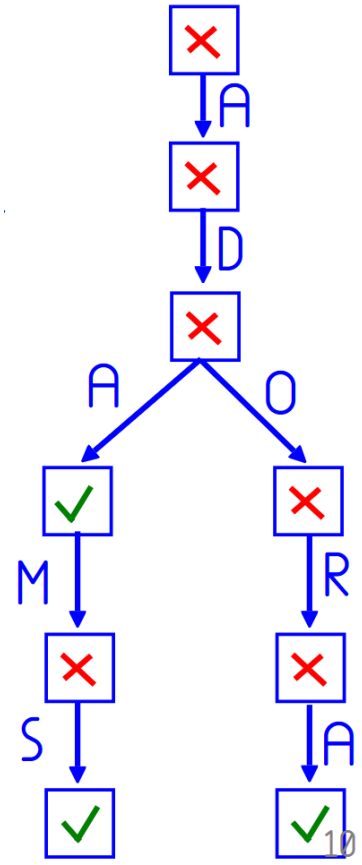


Figure: Example from [Suchanek, 2021].

Efficiently finding words in a data structure: Trie

To find all the named entities in a text: from every character in the text advance until you find a true node or until you cannot continue in the trie.

Adams adores Adora.

Running time: O(textLength X maxDepthTrie)

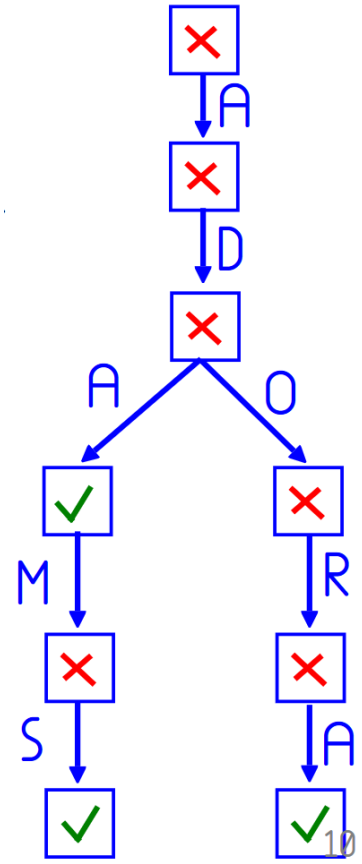


Figure: Example from [Suchanek, 2021].

Coreference

Dr. Marie Curie discovered radium. **She** was married to Pierre Curie, and **the couple** resided in Paris.

Coreference resolution is the task of finding all expressions that refer to the same entity in a text.
Coreference: "She" and "the couple" refer back to [Marie Curie] and [Pierre Curie].

Coreference Method: Assign the pronoun to the closest preceding entity, or the entity in the subject position

Named Entity Classification (NERC)

Once we have found the boundaries of a named entity (NER), we must assign a semantic class to it.
This is also known as **Entity Classification/Typing**.

"Dr. **Marie Curie** discovered radium. She was wed to **Pierre Curie**, and the couple resided in **Paris**."

- [Marie Curie] $\rightarrow$ `PERSON`
- [Pierre Curie] $\rightarrow$ `PERSON`
- [Paris] $\rightarrow$ `LOCATION`

We can be even more specific:
[Marie Curie, Pierre Curie] $\rightarrow$ `SCIENTIST`

Named entity recognition and classification (NERC)

Hearst Patterns: Consider patterns that co-occur with input entity and the desired output class. E.g. "Physicists such as Marie Curie": Marie Curie is a physicist.

1. **Input:** Start with a small set of trusted entities of a given class: $S =$ *(Marie Curie, Pierre Curie)*
2. **Output:** Patterns for this class, and new entities of the class
3. Initialize P with empty set or with pre-specified patterns
4. 1. Pattern discovery:
   1. identify co-occurring phrases with entity $e \in S$ in web corpus;
   2. generalize phrases into patterns ("physicists such as X");
   3. add frequent patterns;
5. 2. Statement expansion:
   1. identify co-occurring entities with patterns $p \in P$ in web corpus;
   2. add frequent entities to S
6. Repeat steps 4 and 5 until no new patterns can be learnt.

Hearst Patterns: Step-by-Step Execution

Discover new entities for the class `Physicist`.

1. Initialize with Seeds: Set $S = \{\text{Marie Curie, Pierre Curie}\}$

2. Pattern Discovery: Search the web for the class name and the seed entities: *"Brilliant physicist such as Marie Curie and Pierre Curie paved the way."*
$\rightarrow$ **Extracted Pattern:** `physicist such as X and Y`

3. Statement Expansion: Search the web using our newly discovered pattern: *"Other physicists such as Albert Einstein and Niels Bohr attended the conference."*

The Result

The algorithm automatically learns that *Albert Einstein* and *Niels Bohr* belong to the class `Physicist`, adds them to set S, and repeats the process!

Advanced NERC: Hierarchical Entity Typing

After NERC, we can a label (e.g., *Marie Curie* → `PERSON` or *Marie Curie* → `PHYSICIST`). In Knowledge Bases, types form a **Taxonomy**.

Taxonomic Typing

Marie Curie is a `Physicist`.
Every `Physicist` is a `Scientist`.
Every `Scientist` is a `Person`.

Why does this matter?

- **Inference:** If we know an entity's specific class, we automatically inherit all parent classes.
- **Relation Constraints:** If a relation like `wonPrize` requires a `Person`, the algorithm knows a `Physicist` is valid, but a `Location` is not.

Entity Canonization/Disambiguation (Entity Linking)

Entities found: [Marie Curie], [Pierre Curie], [Paris]

- *Marie Curie* → Linked to Wikidata ID: **Q7186**
- *Pierre Curie* → Linked to Wikidata ID: **Q37463**
- *Paris* → Linked to Wikidata ID: **Q90** (Capital of France)

- **Canonization Method:** If an entity string matches multiple KB entries (e.g., Paris, France vs. Paris, Texas), match the mention to the KB entity that has the most statements (triples) or if the KB entries have different classes use the class as a filter.
- Alternatively, use surrounding context words to disambiguate.

Entity Canonization/Disambiguation: Context Overlap

If entities have similar number of statements, we must use the surrounding text.

"Dr. Marie Curie discovered radium. She was married to Pierre Curie, and the couple resided in **Paris**."

The Overlap Algorithm:

1. **Context Set:** Extract keywords from the sentence: {**marie curie**, discovered, radium, married, **pierre curie**, couple, resided}
2. **KB Entity 1 (Paris, France):** Extract keywords from its KB description: {france, city, europe, **marie curie**, **pierre curie**}
3. **KB Entity 2 (Paris, Texas):** Extract keywords from its KB description: { texas, city, american, united, states}
4. **Intersect:** Entity 1 shares 2 keywords with the context set, while Entity 2 shares 0. Entity 1 wins!

Advanced Disambiguation: Collective Entity Linking

Sometimes, resolving entities one-by-one fails. We must disambiguate them **collectively** using a Mention-Entity Graph.

The Ambiguity Problem

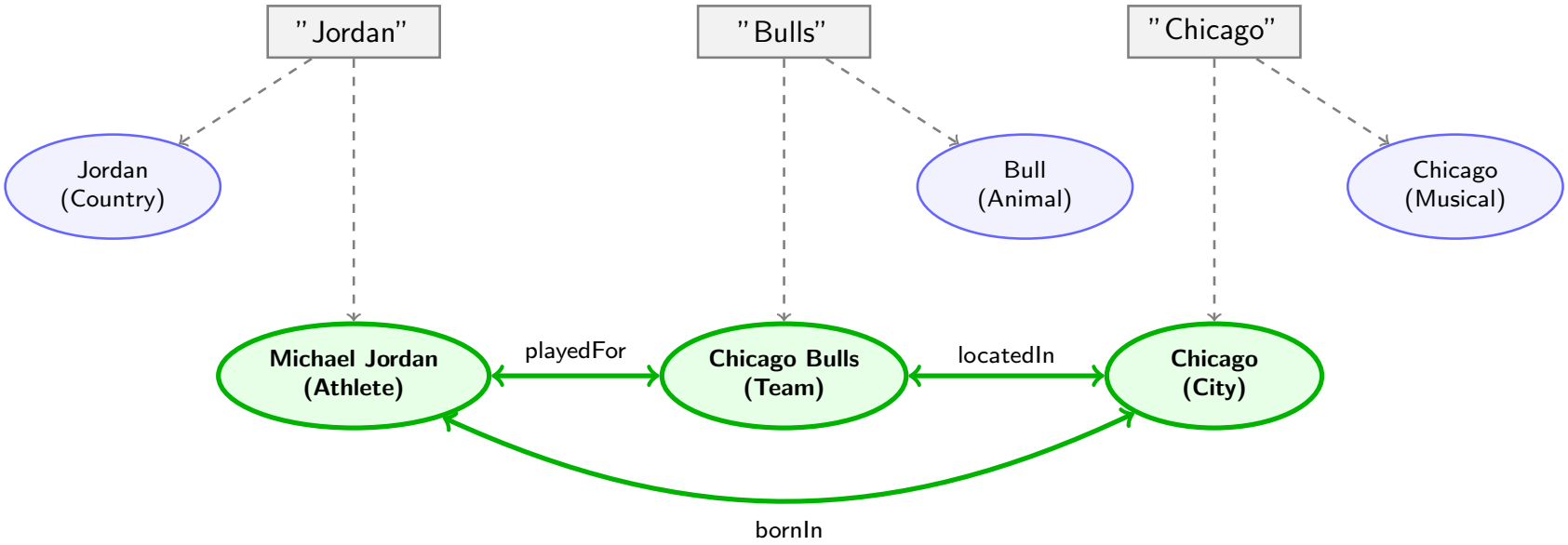
"In 1990, **Jordan** played against the **Bulls** in **Chicago**."

- *Jordan*: Michael Jordan (Athlete) OR Jordan (Country)?
- *Bulls*: Chicago Bulls (Team) OR Male Cows (Animal)?
- *Chicago*: The City OR The Musical?

Graph Solution: We map all possible KB entities for each mention as nodes in a graph. We add edges between entities that share KB relations (e.g., Michael Jordan played for the Chicago Bulls). The sub-graph with the highest density (most interconnected nodes) gives us the correct, collective disambiguation!

Visualizing Collective Entity Linking

By mapping all possible candidates, we look for the **densest sub-graph** created by known KB relations.



- **Gray dashed lines:** Possible entity matches for the text.
- **Green thick lines:** Known relations existing in our Knowledge Base.

6. Relation Extraction & Canonization

She was married to Pierre Curie, and the couple resided in Paris.

- **Extracted Facts:** (Marie Curie, *was married to*, Pierre Curie)
- **Canonized Triples:** (`Q7186`, `spouse`, `Q37463`) and (`Q7186`, `locatedIn`, `Q90`)

- **Relation Extraction Method:** Use predefined extraction patterns (e.g., "X was married to Y") or algorithms like ReVerb that look for verb-preposition patterns between noun phrases.
- **Relation Canonization Method:** Group relations together using string similarity of the predicate phrases (e.g., "is married with", "is the spouse of").

Relation extraction / fact extraction

1. Relations are known. DIPRE [Brin, 1998] algorithm: for a given input relation **r** :

1. Start with a small set of trusted pairs (subject, object) that are in relation **r**: *(Marie Curie, Paris)*
2. Find occurrences of these pairs in sentences.
3. Identify generalized patterns from the sentences fetched in step 2: *X lives in Y, X works in Y*.
4. Use these identified patterns to find more pairs (subject, object): *(Pierre Curie, Paris)*.
5. Repeat steps 2 to 4 until no new patterns can be learnt.

At the end we will have a set of facts and patterns for a given relation r.

DIPRE Algorithm: Step-by-Step Execution

One iteration of DIPRE to extract the `residence` relation.

1. Start with a Seed Pair:
We start with one trusted fact: **(Marie Curie, Paris)**

2. Find Occurrences in Text:
The system searches the web for sentences containing both entities:
"In 1891, Marie Curie moved to Paris to study."
"Marie Curie lived and worked in Paris."

3. Identify Generalized Patterns:
Replace the entities with variables to create extraction rules:
Pattern A: `X moved to Y`
Pattern B: `X lived and worked in Y`

4. Extract New Pairs:
Search the web using the new patterns: *"Albert Einstein moved to Princeton."* $\rightarrow$ **(Albert Einstein, Princeton)**

Relation extraction / fact extraction

2. Relations are not known. The ReVerb algorithm [Fader et al., 2011] has two steps: relation extraction and argument extraction.

A relation is of the form **V|VP|VW*P** where
V=(aux verb|verb) particle? adverb?
W=(noun|adj|adv|pron|det)
P=(prep|particle)

Example: verb (*walks*), a verb followed by a preposition (*located in*), or a verb followed by nouns, adjectives, or adverbs ending in a preposition (*has atomic weight of*).

The arguments of the relations are the nearest noun phrases to the left and to the right of the relation (My best friend walks every day.).

ReVerb Step-by-Step Execution

Let's trace the algorithm on a new sentence.

The Input Sentence

"Carbon has an atomic weight of 12."

1. POS Tagging & Chunking:

`[Carbon / NounPhrase] [has / VERB] [an / DET] [atomic / ADJ] [weight / NOUN] [of / PREP] [12 / NounPhrase].`

2. Relation Extraction (The Regex):

The algorithm scans the tags. It finds a verb (*has*) followed by an adjective and a noun, ending in a preposition (*of*). This perfectly matches the `V W* P` pattern!

→ **Extracted Relation:** *"has an atomic weight of"*

3. Argument Extraction:

The algorithm looks for the nearest noun phrase to the left (*Carbon*) and the nearest to the right (*12*).

Why ClausIE? The Limits of Shallow Relation Extraction

ReVerB Limitations:

- Algorithms like ReVerb look for strict verb-preposition patterns between adjacent noun phrases.
- They struggle with long, complex sentences where the distance between the subject and the object is stretched by nested clauses or modifiers.
- *Example:* "The scientist who discovered radium won the prize." A shallow left-to-right scan might completely miss *(scientist, won, prize)*.

The ClausIE Approach: Dependency to Clauses

ClausIE (Clause-based Open IE) exploits linguistic knowledge about English grammar to detect clauses.

- It utilizes **dependency parsing** and POS tagging to map out the entire grammatical structure first.
- By breaking a complex sentence into a set of simpler, coherent clauses, it bypasses the "long distance" problem entirely.

How do we find Clauses? The Dependency Tree

Dependency parsing determines the syntactical structure of a sentence by finding words that act as a single unit. But how does this give us a "clause"?

In a dependency tree, **verbs act as the root** of a clause. The outgoing arrows from the verb point directly to its core arguments.

Mapping Dependencies to Clause Parts

Sentence: "The brilliant scientist quickly discovered radium."

- **The Root (Verb):** The algorithm searches for the root node → *"discovered"*.
- **The Subject:** It follows the `nsubj` (nominal subject) edge from the verb → *"scientist"*.
- **The Object:** It follows the `dobj` (direct object) edge from the verb → *"radium"*.

Handling Embedded Clauses

What happens when a sentence contains a clause *inside* another clause? The algorithm simply looks for multiple verb roots in the tree!

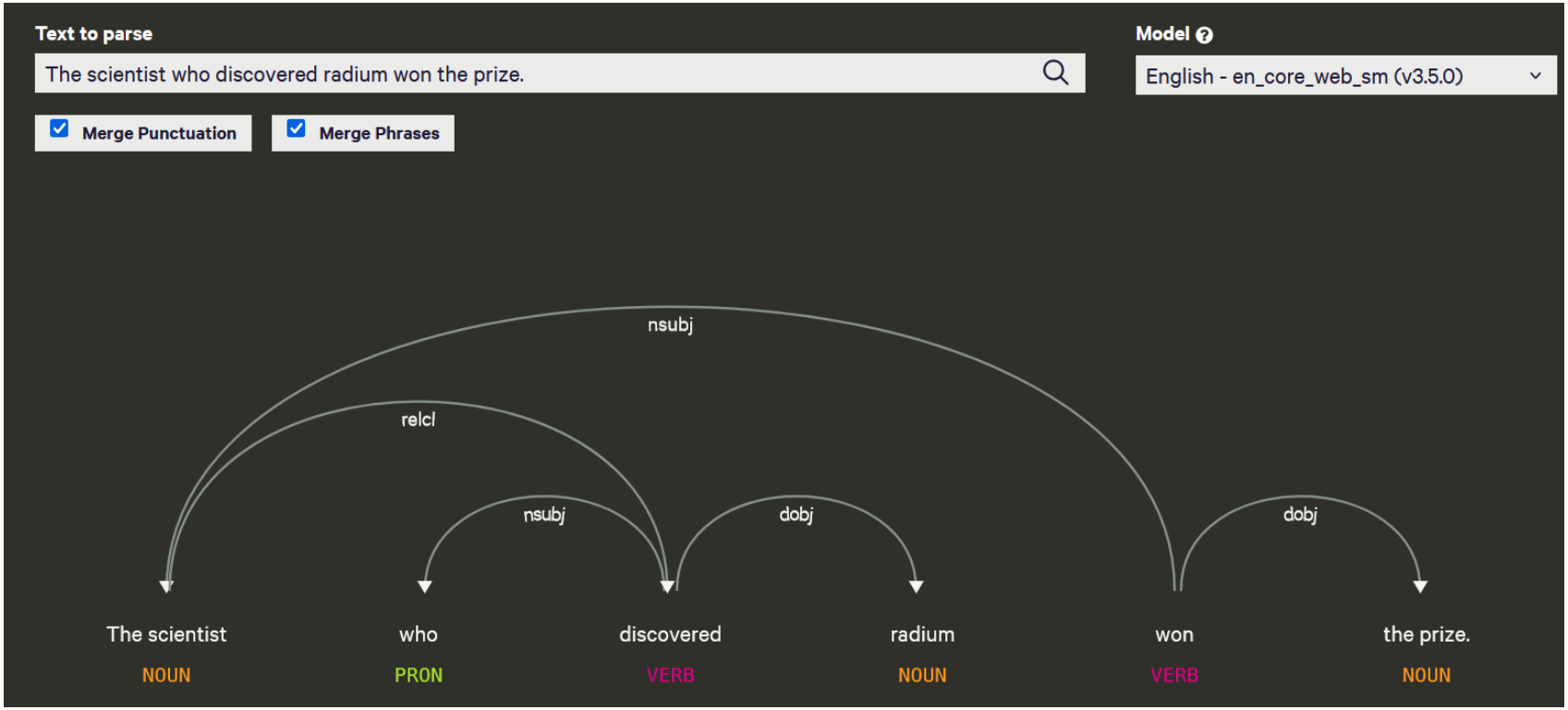


Figure: Caption

Executing the Extraction Algorithm

The algorithm processes every verb it finds in the tree to generate independent propositions.

Iteration 1: Main Clause

- **Target Verb:** *won*
- **Find Core Arguments:** Subject is *the scientist*, Object is *the prize*.
- **Result 1: (The scientist, won, the prize)**

Iteration 2: Embedded Clause

- **Target Verb:** *discovered*
- **Find Core Arguments:** Subject is *who*, Object is *radium*.
- **Resolve Reference:** The algorithm sees *discovered* is a relative clause (`relcl`) attached to *the scientist*. It replaces *"who"* with *"The scientist"*.
- **Result 2: (The scientist, discovered, radium)**

The Problem: Paraphrased Relations

After running an extraction algorithm (ReVerb, Clausie), we extract many different predicate phrases that express the exact same meaning.

To build a clean Knowledge Base, we must group them into a single canonical relation.

The Synonym Problem

Extracted Surface Forms → Canonical Relation

- "is situated in", "is part of", "is located in" → `locatedIn`
- "is married with", "is the spouse of", "is the wife of" → `spouse`

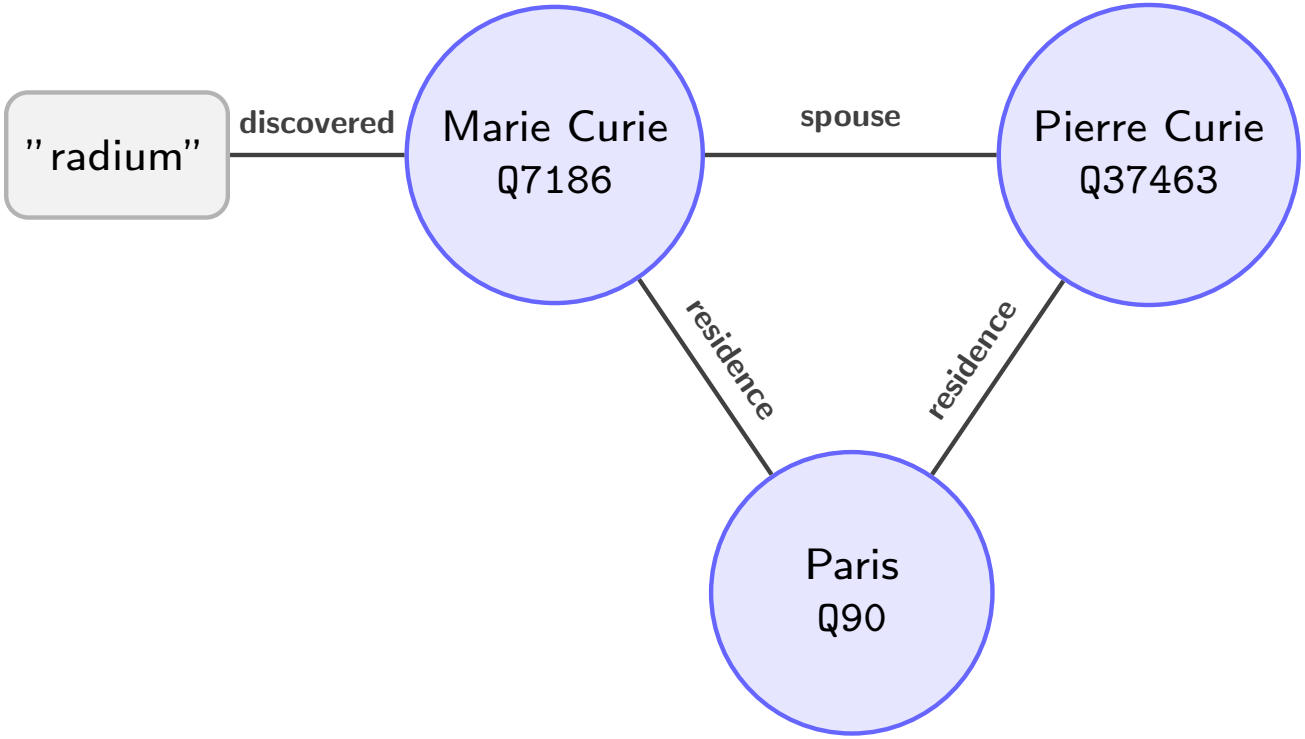
How to Compare Relations? Feature Extraction

To group relations, we must measure how "similar" they are mathematically. We use several features:

- **1. String Similarity:** How closely do the characters match? (e.g., "is married **to**" vs. "is married **with**").
- **2. Entity Overlap:** Do they connect the same entities? If (*Marie*, R1, *Pierre*) and (*Marie*, R2, *Pierre*) frequently occur, R1 and R2 are likely synonyms.
- **3. Type Signatures:** Do they demand the same entity types? (e.g., `spouse` requires Subject=`Person`, Object=`Person`).
- **4. Context Similarity:** Do these relations appear in sentences with similar surrounding vocabulary?

Final Output: The Extracted Knowledge Graph

After applying Segmentation, POS Tagging, NER, Coreference, Entity Linking, and Relation Canonization, we transform unstructured text into structured triples.



Evaluating the quality of a KB

Precision, recall, F1

Precision and recall for:

- for a set of entities that populate a given class;
- for all entities of a given type;
- for the set of classes known for a given entity;
- for the set of property statements about a given entity;

We assume the KB contains a set of statements S we want to evaluate, and a ground-truth set GT.

$$\text{Precision}(S) = \frac{|S \cap GT|}{|S|}$$

$$\text{Recall}(A) = \frac{|S \cap GT|}{|GT|}$$

Creating the ground truth

Creating a ground truth: depending on what we evaluate, we would need a very comprehensive set (e.g. all the songs of a singer, all the people that are singers).

In practice, we often resort to proxies for the grounth truth.

Precision: We select a (random) subset of elements from the KB, and for each of them, we decide if it is correct or not.

Recall: Sampling is not ideal, as the goal is capturing all knowledge of interest. We instead collect a small but complete ground-truth set, for example, all songs by a singer.

Rule-Based Extraction: Precision vs. Recall

When writing extraction rules (like Regular Expressions or Hearst Patterns), we face a fundamental trade-off between **Precision** (Accuracy) and **Recall** (Coverage).

Strict Rules

Pattern: `[A-Z][a-z]+ was born in [A-Z][a-z]+`

- High Precision: Rarely extracts garbage.
- Low Recall: Misses "Marie Curie, born in Warsaw" because of the comma.

Loose Rules

Pattern: `* born in *`

- High Recall: Catches almost every birth fact.
- Low Precision: Extracts garbage like "Poverty is born in ignorance."

Knowledge Engineers must carefully balance these rules based on whether the KB prioritizes being 100% correct or 100% comprehensive.

Knowledge Base Completeness

The Open-World Assumption (OWA) postulates that if a statement is not in the KB, it may nevertheless be true in the real world. Its truth value is unknown. In other words, absence of evidence is not evidence of absence [Weikum et al., 2020].

The Local Completeness Assumption (LCA), aka. Partial Completeness Assumption or Local Closed-World Assumption, for entity S in the KB asserts that if, for some property P, the KB contains at least one statement (S, P,O), then it contains all statements for the same S and P. For example, if we know one of a person's children, then we know all of them; S is complete with regard to P [Weikum et al., 2020].

Summary

Information extraction offers tools to **construct graphs from textual data** using named entity recognition and relation extraction.

Two broad approaches for a IE task:

- Knowledge engineering: find important features that can help constructing rules or patterns. Features should be given by domain experts. **Very hard to cover everything, language is rich and ambiguous.**
- Statistical methods that learn from annotated corpora. Require a large dataset with human annotations. **These techniques perform the best.**

References I

Brin, S. (1998).
Extracting patterns and relations from the world wide web.
In International workshop on the world wide web and databases, pages 172–183. Springer.

Fader, A., Soderland, S., and Etzioni, O. (2011).
Identifying relations for open information extraction.
In Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, pages 1535–1545, Edinburgh, Scotland, UK. Association for Computational Linguistics.

Suchanek, F. (2021).
Information Extraction.
https://suchanek.name/work/teaching/topics/.
[Online; accessed April 2021].

Weikum, G., Dong, L., Razniewski, S., and Suchanek, F. M. (2020).
Machine knowledge: Creation and curation of comprehensive knowledge bases.
CoRR, abs/2009.11564.